

Statistical scraping in Austria

Update for WEB.FOSS-AT, SSIG3-Meeting

Bernhard Meindl, Alexander Kowarik, Johannes Gussenbauer, Eileen Vatheuer
Center Methods (CM)

Porto, 16.09.2026



Overview



Overview

- Progress and current status of webscraping software in R ([taRantula](#))
- Progress and current status of use cases

The background of the slide is a photograph of a modern building's interior, featuring multiple levels with glass railings and potted plants. A semi-transparent blue overlay covers the left and central portions of the image. On the right side, there is a vertical strip showing a closer view of the building's glass facade and a white structural column.

Webscrapping Software: R Package taRantula

Agenda

1. **Overview:** what `taRantula` is (architecture)
2. **Configuration:** `R6` parameter manager, YAML, validation
3. **Search:** finding candidates URLs (Google / Brave APIs, OWI corpus)
4. **Scraping:** the `Ur1Scraper` class: engines, validation, parallelism, storage
5. **Data access:** results, links, logs, SQL, regex extraction
6. **Infrastructure:** Selenium Grid with Docker
7. **Summary:** design trade-offs and limitations

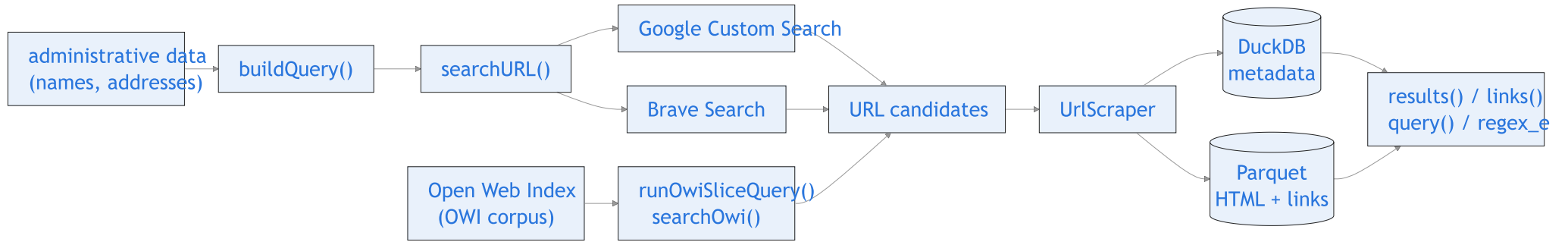
Overview



What is taRantula?

- R package by STAT for **large-scale web scraping** in statistical production
- Typical task:
 - derive company/institution websites from administrative data,
 - crawl URLs
 - extract structured information (e-mails, UID/VAT numbers, links)
- Important concepts:
 - **Persistence**: intermediate results are stored on disk (DuckDB + Parquet) → jobs survive crashes
 - **Parallelism**: multiple browser/HTTP workers via **future**, driven against a Selenium Grid
 - **Compliance**: **robots.txt** enforcement, domain reachability, politeness delays

Architecture



Two Main Feature-Sets

Search — which URLs?

- `buildQuery()`, `searchURL()`
- GCS (Google) / Brave Search supported
- rate limiting, backoff, resume
- `runOwiSliceQuery()`, `searchOwi()`: keyword search in an existing web-archive corpus (Parquet via DuckDB)

Output: table of candidate URLs with ranking positions

Scraping — fetch & parse

- `UrlScraper` R6 class
- Choices: Selenium (JS) or `httr2` (static)
- pre-validation: domain check, `robots.txt`
- Parallel workers, snapshots, DuckDB + Parquet storage

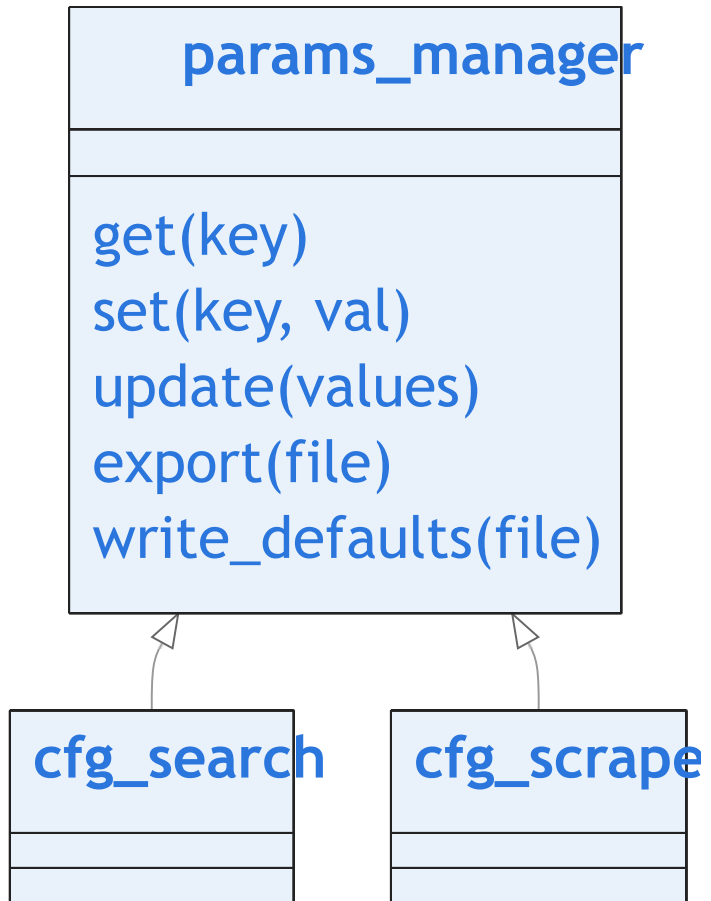
Output: HTML sources, link graph, logs, regex hits

Both Feature-Sets share `R6`-based, validated configuration objects.

Configuration



R6 Class Hierarchy



Get, Set, Export

```
1  cfg <- paramsScraper(project = "census_scrape")
2
3  # nested access: "$-syntax" or character vector
4  cfg$get("selenium$host") #> "localhost"
5  cfg$get(c("selenium", "port")) #> 4444L
6  cfg$set("selenium$port", 4445) # validates on write
7
8  # deep merge of (nested) named lists
9  cfg$update(list(robots = list(check = FALSE)))
10
11 # portability / reproducible pipelines
12 cfg$export("my_config.yaml") # current state
13 cfg$write_defaults("template.yaml") # package defaults
14
15 new_cfg <- paramsScraper(config_file = "my_config.yaml")
```

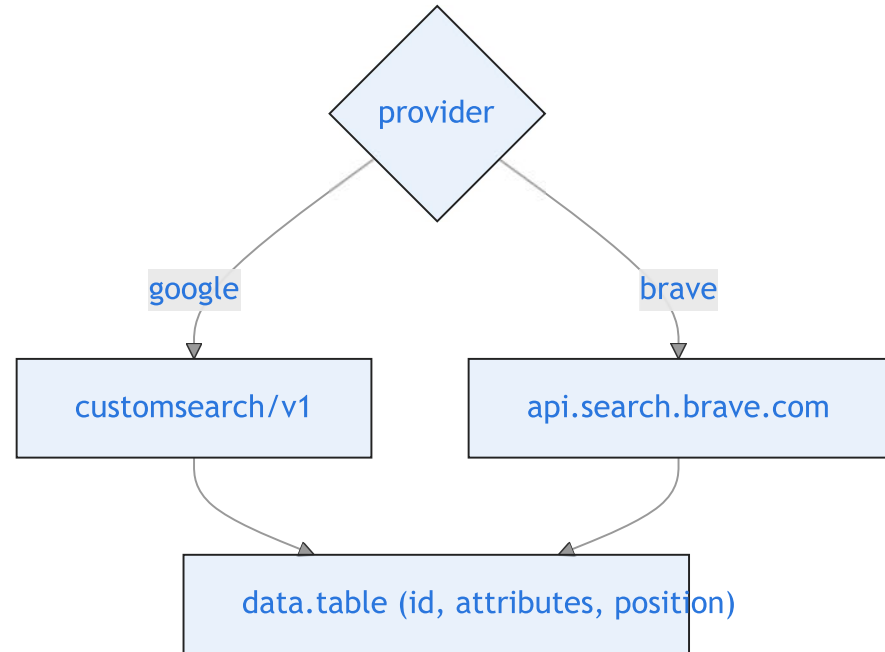
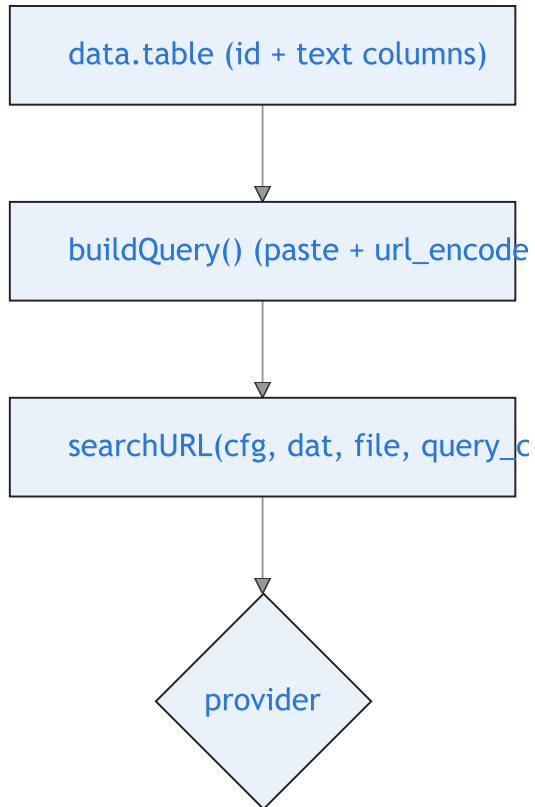
- `$show_config()` returns the full list

The image shows a modern building interior with multiple levels, balconies, and potted plants. A semi-transparent blue overlay covers the left and center portions of the image. On the right, a white vertical column and a glass railing are visible. A white search bar is positioned at the bottom center of the image.

Search

Search

Search Pipeline



- one API request per row of **dat**
- results contain **entity id** and **ranking position**
- allows to set what to keep (e.g **title** / **link**)

buildQuery()

```
1 dat <- data.table(  
2   ID = c(1, 2, 3),  
3   EnterpriseName = c("Name1", "Name2", "Name3"),  
4   EnterpriseAddress = c("Address1", "Address2", "Address3")  
5 )  
6  
7 dat[,  
8   Query1 := buildQuery(.SD),  
9   .SDcols = c("EnterpriseName", "EnterpriseAddress")  
10 ]  
11 dat[, Query2 := buildQuery(.SD), .SDcols = "EnterpriseAddress"]
```

- concatenates the selected columns per row, trims, applies `urltools::url_encode()`
- tags the vector: `attr(q, "query_attr") == "built_encoded_query"`
- `searchURL()` **aborts** on any query column lacking this tag; this guards against raw, unencoded input

Google Custom Search

```
1 Sys.setenv(  
2   SCRAPING_APIKEY_GOOGLE = "****",  
3   SCRAPING_ENGINE_GOOGLE = "My_Engine"  
4 )  
5  
6 cfg <- paramsGoogleSearch(  
7   path = "~/path/to/my/project",  
8   scrape_attributes = c("title", "link", "displayLink", "snippet")  
9 )  
10 cfg$set("query_col", c("Query1", "Query2"))  
11 cfg$set("file", c("File1.csv", "File2.csv"))  
12  
13 runGoogleSearch(cfg = cfg, dat = dat)
```

Brave Search and URL Blacklist

```
1 Sys.setenv(SCRAPING_APIKEY_BRAVE = "****")
2
3 dat <- data.table(id = 1, term = "st-georgen-kreischberg")
4 dat[, query := buildQuery(.SD), .SDcols = "term"]
5
6 cfg <- paramsBraveSearch(
7   id_col = "id",
8   query_col = "query",
9   blacklisted_urls = "https://www.st-georgen-kreischberg.gv.at/",
10  scrape_attributes = c("title", "link", "displayLink", "snippet"),
11  verbose = FALSE
12 )
13
14 urls <- searchURL(
15   cfg = cfg,
16   dat = dat,
17   file = NULL,
```

Rate Limiting and Resilience

```
1 cfg_search_defaults <- paramsBraveSearch()
2 cfg_search_defaults$get("max_query_rate") #> 100
3 cfg_search_defaults$get("max_queries") #> 10000
4 cfg_search_defaults$get("save_every_n") #> 500
```

- **rate window:** every `max_query_rate` requests the loop sleeps to stay within `n` requests / 100 s; on `max_queries` per 24 h it sleeps until reset
- failed API calls are retried with **exponential backoff** (sleep 1, 2, 4, ... s; up to 16 attempts); `HTTP ≥ 400` treated as an error
- incremental persistence: results CSV rewritten every `save_every_n` queries
- resume: with `overwrite = FALSE`, an existing file is loaded and already-searched ids are skipped

Context: OpenWebSearch.eu

- **OpenWebSearch.eu** : EU-funded research project building an open, diverse European web-search ecosystem
- the **Open Web Index (OWI)**: a continuously updated, openly accessible web corpus
 - crawling → daily slices → preprocessing & semantic enrichment → indexing
- motivation: web search is critical infrastructure dominated by few commercial gatekeepers; the OWI could open the web as a resource for research, innovation, and public statistics (AI, web analytics, ...)
- access: make use of the **owilix** client; data is retrieved as **query slices**, filtered by route (date range), collection, and fields (e.g. **url_suffix**)
- Book & tutorials: *The Open Web Search Book*

taRantula integrates this workflow into **R**.

OWI Corpus Search (1)

Second search mode: not the live web, but the **Open Web Index** corpus

```
1 # slice the corpus with the external `owi` CLI
2 # Example: the Austrian legal-pages
3 pq <- runOwiSliceQuery(
4   route = "all:latest/collectionName=legal",
5   collection = "austrian_legal_pages"
6 )
7
8 # keyword search over the resulting parquet files
9 hits <- searchOwi(
10   pq$parquet_path,
11   keyword = "Impressum",
12   field = "main_content",
13   exclude_domains = c("example.com"),
14   limit = 500
15 )
```

OWI Corpus Search (2)

- `runOwiSliceQuery()` builds and runs the `owi query slice` command
 - default filter `url_suffix='at' AND valid=true`
 - returns the `*.parquet` glob of the most current dataset
 - `dry_run = TRUE` allows to preview the command
- `searchOwi()` runs a generated query on an in-memory DuckDB via `read_parquet()`
 - allows literal (`strpos`) or regex (`regexp_matches`) matching
 - optional features:
 - case-insensitivity
 - domain exclusion
 - column pruning

OWI Corpus Search (3)

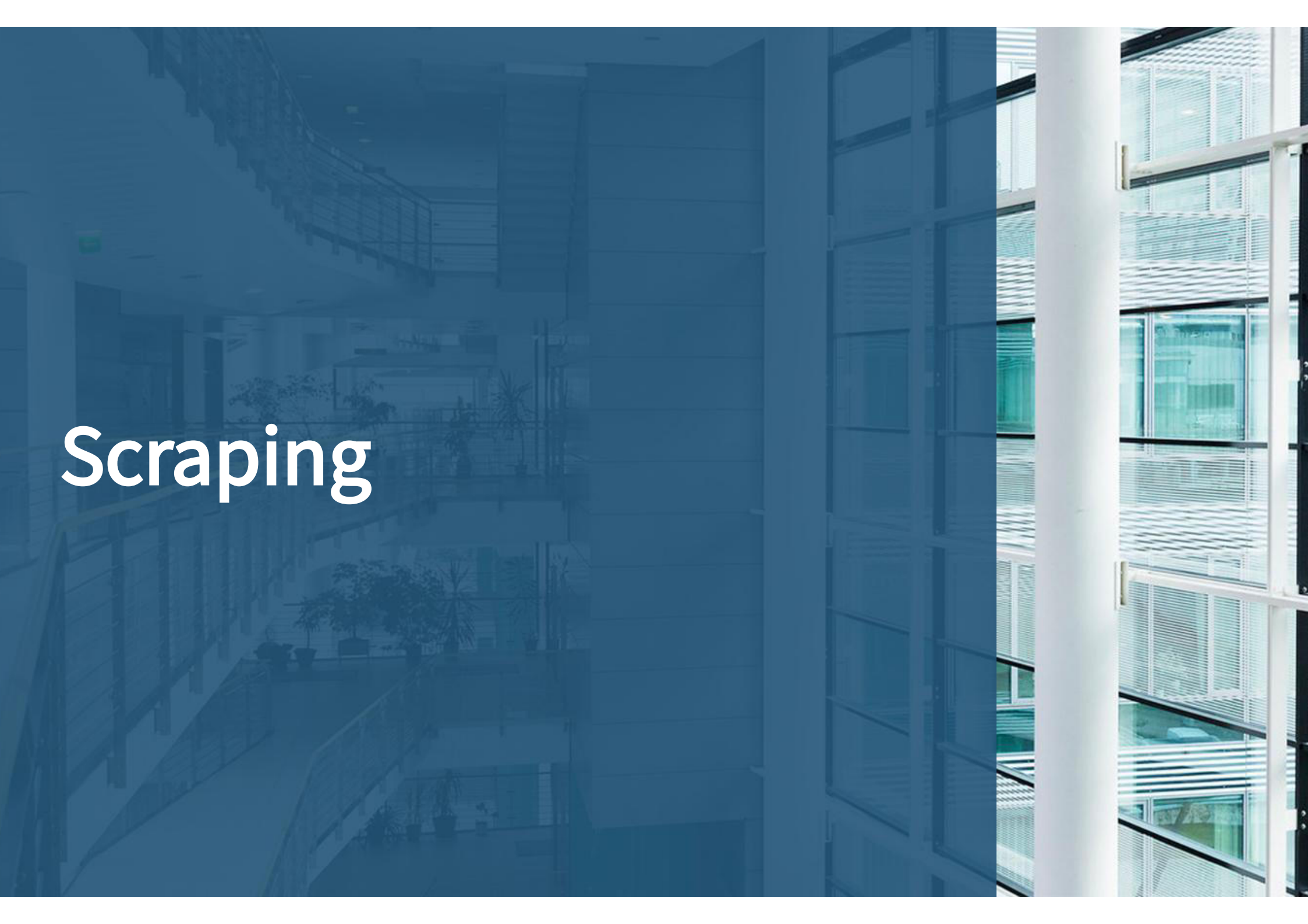
- all legal slices in a certain timespan can be filtered according to the URL suffix and downloaded.
- How long should this time span be?

```
1 pq_slice <- runOwiSliceQuery(  
2   route = "all:2026-01-01..2026-08-13/collectionName=legal"  
3 )
```

- For Austrian enterprises, the top 3 domains are:
 - at
 - de
 - com
- Just download the full legal slices?

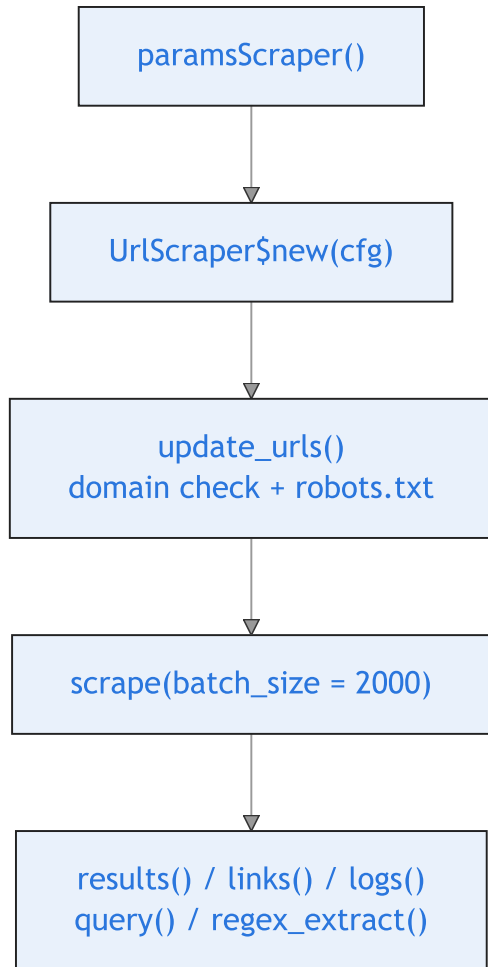
Evaluation Search Results

- Sample of 500 enterprises with an URL
- Brave finds a URL for 98%
- BUT: only about 10% of the URLs match
- Some manual inspection of the mismatch:
 - initial URL is not working anymore
 - initial URL and new URL lead to the same website
 - initial URL is wrong (now?)
- TODOs:
 - Make a full comparison, either by scraping the URLs or by manual inspection
 - Include OWI search in the comparison.

The image shows a modern building with a glass facade, partially obscured by a blue-tinted overlay. The building has multiple floors with balconies and large windows. The word "Scraping" is written in white text on the left side of the image. On the right side, there is a vertical strip showing a closer view of the building's glass and metal structure.

Scraping

UrlScraper Lifecycle



Constructor (`initialize()`):

Minimal Working Example

```
1 library(taRantula)
2
3 cfg <- paramsScraper()
4 cfg$set("selenium$host", "localhost")
5 cfg$set("selenium$port", 4444L)
6 cfg$set("selenium$workers", 3)
7 cfg$set("urls", c("https://www.statistik.at", "https://r-project.org"))
8
9 s <- UrlScraper$new(cfg)
10 s$scrape()
11
12 emails <- s$regex_extract("[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+")
13 results <- s$results()
14 s$close()
```

- after `scrape()`, everything is `SQL` / `data.table` work on persisted data
- re-entrant: `update_urls()` + `scrape()` again any time:
 - duplicates (within input **and** against the database) are pre-filtered

Two engines

	Selenium (default)	httr2
JavaScript rendering	yes (full browser)	no
content	rendered DOM after load	raw response body
redirect info	browser's final URL	<code>resp\$url</code>
cost	browser session on the grid	one <code>GET</code> , <code>30s</code> timeout
use when	dynamic sites	static sites, bulk speed

```
1 cfg$set("selenium$use_selenium", FALSE) # same code, httr2 fallback
```

- Chrome capabilities (eg. timeouts or user-agent) can be adjusted via config
- engine switch happens inside `.create_sid()`; worker code is identical,
- redirect detection: `clean_url(request) != clean_url(final)` → stored as `url_redirect` in `results`

Pre-Scrape Validation

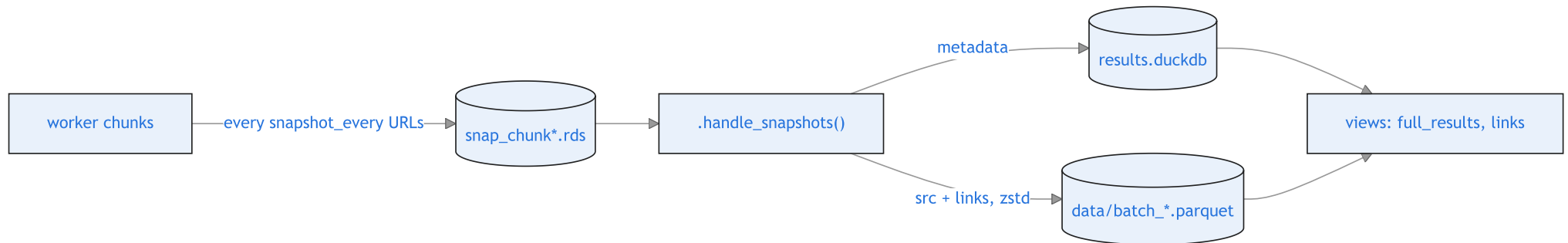
- Runs once per new URL inside `update_urls()` **before** any worker starts.
- Workers only process pre-validated `todo` URLs
 - **domain check** (`.handle_domaincheck()`): parallel `HEAD` requests
 - unreachable domain → all its URLs become `failed_domain`
 - **robots.txt** (`.handle_robots()`): fetched per *new* domain via `robotstxt`
 - cached in the `robots` table → pending URLs are validated against the cached rules

<code>results.status</code>	meaning
<code>todo</code>	queued, pre-validated, ready for workers
<code>success</code>	scraped; content available in <code>parquet</code>
<code>failed_domain</code>	domain not reachable
<code>failed_scraping</code>	fetch failed after retries

Parallel Workers and Resilience

- `scrape(batch_size = 2000)` orchestration:
 - `future::multisession` with `selenium$workers`; each batch split into equal chunks, one per worker (`future_lapply`)
- per-worker loop (`.worker_scrape()`):
 - **two-pass strategy**: each URL attempted once; failures go to a retry queue (up to 2 further attempts, refreshed session, longer pauses)
 - Sessions are probed every 50 URLs and restarted after 5 consecutive failures
 - Random sleeps are performed after each scraping attempt
 - workers **never touch DuckDB**, they write `.rds` snapshots and per-worker progress logs; the main process ingests after each batch
 - overall progress via `progressr` + `cli`

Hybrid Storage



DuckDB tables

- **urls**: registered URLs, **level**, **added_at**
- **results**: url, url_actual, url_redirect, status, **file_path**, robotscheck_allowed, scraped_at
- **logs**: per-URL progress entries
- **robots**: cached **robots.txt** per domain

DuckDB views over Parquet

- **full_results**: joins **results** with **{data_dir}/*.parquet** → adds **src** and **links**
- **links**: **unnest(links).href/.label**
- recreated after every batch

Raw HTML stays out of the database: compressed **parquet**, **SQL**-joinable on demand.

Snapshots and Recovery

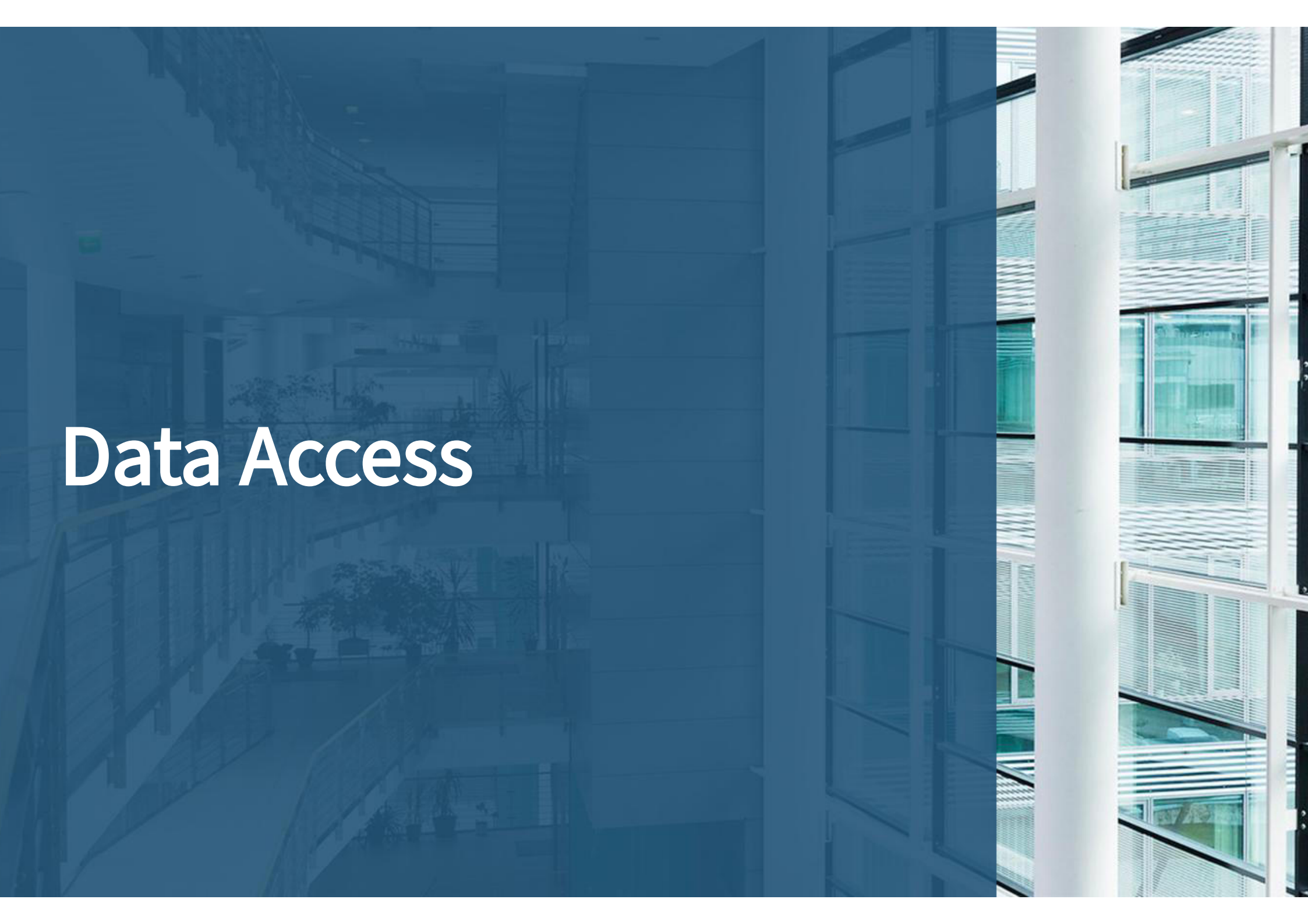
- every `selenium$snapshot_every` URLs, a worker flushes its buffer to `snap_chunk{id}_{timestamp}.rds` and continues with an empty one
- `.handle_snapshots()` (init, after each batch, at `close()`):
 - Batch-reading of snapshots and writing compressed `parquet` (`url`, `scrapped_at`, `src`, `links`)
 - inserts metadata into `results` in one transaction
 - refreshes views
 - deletes the snapshots; failed batches are moved to `snapshot/_failed/`
- crash recovery: restart `UrlScraper$new(cfg)` with the same `base_dir` / `project`
 - pending snapshots are imported, previously written `parquet` files remain valid; only the small unflushed buffers are at risk
- `s$close()`: final snapshot/log import, removes `snapshot/` + `progress/`, shuts down the DB connection

Link Extraction and Filtering

```
1 extractLinks(html, baseurl = "https://example.com/impressum")
2 #>      href label source_url scraped_at
```

- `extractLinks()`: parses `a`, `area`, `base`, `link` elements; resolves relative URLs against the base; normalizes link text (`label`)
- `clean_url()` normalization: force `https://`, drop `#fragment`, drop trailing `/` → consistent join keys
- `check_links()` keeps only **same-domain** URLs and drops:
 - ~**100** asset extensions (images, video, audio, fonts, CSS, PDF/Office, ...)
 - pure anchor links on the current page
 - duplicated path+parameter combinations
- stored per page in the `parquet links` column; exposed via the `links` view

Data Access

The image features a modern building interior with a blue overlay. On the right side, there is a glass-walled elevator or staircase. The text "Data Access" is prominently displayed in white on the left side of the image.

results, links, logs, query

```
1 res <- s$results() # joins parquet: includes src
2 meta <- s$results(with_src = FALSE) # metadata + file_path only
3 lnk <- s$links() # edges + computed hierarchy level
4 log <- s$logs()
5
6 s$query(
7   "SELECT status, COUNT(*) AS n
8     FROM results GROUP BY status"
9 )
```

- all methods accept a raw SQL **filter** (WHERE condition without **WHERE**):
`s$results(filter = "url LIKE 'https://example.com/%'")`
- `with_src = TRUE` materializes the **full_results** view — on-the-fly join with the Parquet archive; **FALSE** is much faster for summaries
- `links()` computes hierarchies; depth per target URL, starting at the registered root URLs (**urls** minus all link targets)

Regex Extraction

```
1 vat <- s$regex_extract(  
2   pattern = "ATU[0-9]{8}", # Austrian UID / VAT numbers  
3   filter_links = c("imprint", "contact"),  
4   ignore_cases = TRUE  
5 )  
6 #>           url      pattern  
7 #> 1: https://example.com/imprint  atu12345678
```

1. select links whose `href` or `label` matches `filter_links` (case-insensitive `%ilike%`)
2. restrict to successfully scraped pages containing one of those links
3. strip HTML to visible text (`parse_HTML()` removes `script`, `style`, `noscript`)

Queue Management and Status

```
1 s$url_info
2 #> nr_urls  nr_scraped  nr_failed_domaincheck
3 #> nr_failed_scraping  nr_todo  nr_blocked
4
5 s$urls_todo # pending URLs (status "todo")
6 s$update_urls("https://ec.europa.eu/eurostat")
7 s$remove_urls("https://old.example.org") # only "todo" entries
8 s$stop()
```

- active bindings query DuckDB in real time (`get_url_statistics`, `get_todo_urls`)
- `update_urls(force = FALSE)`: skips URLs already scraped; `force = TRUE` re-queues them
- `remove_urls()` deletes **only pending** entries (scraped or blocked URLs remain untouched)
- every mutation prints a `cli` summary of added / done / blocked counts

Infrastructure



Selenium Grid with Docker

```
1  services:
2    hub:
3      image: selenium/hub:4.34.0
4      ports: ["4442-4444:4442-4444"]
5    node-chrome-1:
6      image: selenium/node-chrome:138.0
7      depends_on: [hub]
8      environment:
9        - SE_EVENT_BUS_HOST=hub
10       - SE_EVENT_BUS_PUBLISH_PORT=4442
11       - SE_EVENT_BUS_SUBSCRIBE_PORT=4443
12       - SE_NODE_OVERRIDE_MAX_SESSIONS=true
13       - SE_NODE_MAX_SESSIONS=3
14      shm_size: 2g
15      deploy:
16        resources:
17          limits: { cpus: "2.5", memory: 6G }
```

Summary



Design Trade-offs

- **hybrid DuckDB + Parquet storage:** relational metadata stays small and `SQL`-queryable; multi-MB HTML goes to columnar compressed `parquet`, joined on demand via views
- **validated R6 configuration:** fail fast at `$set()` instead of inside a crashed worker three hours later;
- **pre-scrape validation phase:** robots/domain checks moved out of the parallel loop: no redundant network round-trips per URL
- **snapshot-based resumability:** workers write to files, one process owns the database; crash = re-instantiate, done
- **graceful worker termination:** stop-file signal, no corrupted database, no orphaned browser sessions
- tested with a `testthat` suite: scraper config validation, search result contract (`id` + attributes + `position`), OWI query building; CI runs `R CMD check`

Limitations and Assumptions

- Selenium support is for now **Chrome-only** (validated `browser = "chrome"`); the grid must be provisioned externally (Docker)
- search queries hardcode the AT/de context (`gl=at, lr=lang_de, country="AT"`) — Austrian-administrative-data focus
- OWI path depends on the external `owi` CLI and the `owilix_path` environment variable
- `regex_extract()` loads all matching documents into memory — chunk via `filter` for very large crawls
- rate limiting is a blocking `Sys.sleep()` inside one R session — search jobs are sequential by design
- `parquet` files accumulate per batch; no compaction/vacuum step yet

Resources & Questions

- Vignettes: *Getting Started with taRantula Configuration Management with params_manager*
- Source & docs
- external: Google Custom Search API, Brave Search API, Selenium Grid documentation, *The Open Web Search Book* (OpenWebSearch.eu / OWI)
- Perhaps try **Playwright** as an alternative to Selenium

Use Cases



Scraping wine prices



Scraping wine prices

- **Aim:** collect wine prices from websites of Austrian winemakers
- **Why:** used as proxy for the agricultural producer price index
- **Rough outline of actions:**
 1. Start with units from Statistical Business Register who run a vineyard
 2. Search their URLs and look for Webshop
 3. Scrape/Extract price list of wines
 4. Estimate average price for different wines/regions/types

Scraping wine prices

- Starting with fields from SBR:
 - Name, Address, *Area of vinyard in m²*

- Identified URLs

# Enterprise	# Websites found	Share (%)	weighted Share (%)
9586	2230	23.26	44.06

Scraping wine prices

- Attributes to collect: Vintage, Price, volume, Red/White, *Quality*: example
- Focus-Scrape on subpages with potential online shop/ wine lists / pdfs of wines
- Collecting wine prices with LLMs
 - feed raw HTML
 - prompt contains large list of rules
 - currently SSPCloud for testing (gpt-oss:120b)
- ▶ expand for prompt
- Using LLMs to collect and process webcontent?

Wine Price Extraction Approach

- **Model:** `gpt-oss:120b` via SSPCloud
- **Input:** Raw HTML from scraped webshop pages and PDF wine lists
- **Output:** Structured JSON with wine attributes
- **Extraction Process:**
 1. **Shop Detection:** LLM checks if page contains valid wine shop
 2. **Product Definition:** Single wines (no sets/bundles), still wine without carbonation
 3. **Attribute Extraction:** For each wine, extract:
 - Wine name, grape variety, color (White/Red/Rosé)
 - Vintage year, quality level, price, bottle volume
 4. **Quality Classification:** Hierarchical rules for Austrian wine quality

Scraping wine prices - Results

- **URL Detection:** Identified websites for vintners from SBR

Website Detection

# Enterprise	# Websites
1075	1039

- **Price Analysis by Volume and Wine Type:**

Mean Prices by Volume (EUR)

Volume	NA	Red	White
NA	15.09	11.56	12.72
0.75L	15.31	13.51	13.26
1L	6.75	7.05	5.79
Other	24.11	24.49	25.82

Price Analysis by Quality Group

- Price Analysis by Quality Group and Wine Type:

Mean Prices by Quality Group (EUR)

Qualitaet_gruppe	NA	Red	White
Anderer Wein	15.49	13.00	12.02
Landwein	5.15	7.85	10.93
Qualitaetswein u Praedikatswein	12.31	13.77	14.09
Unknown	21.18	11.59	21.54

NACE Classification

The image features a modern building interior with a blue overlay. The text "NACE Classification" is prominently displayed in white. The background shows a multi-story building with balconies and large windows, viewed from an elevated position. The blue overlay covers most of the image, with a vertical strip on the right showing a clearer view of the building's facade and a balcony railing.

NACE Classification

- NACE classification using text from enterprise website was already thoroughly investigated during WIN (WP3/UC5)
- Massive work load on re-classification with the NACE revision
- Re-classification of specific NACE-Group

NACE2008

G 47 ~ retail trade and specifically online shops
overall contains 40 000 - 50 000 enterprises

NACE2025

G 47.11 to G 47.83
Retail Sale for food, textiles, IT equipment , ...
About 37 5-digit codes

NACE Classification

- Rough outline of actions:
 1. Start with units from Statistical Business Register
 2. Search their URLs and look for Webshop
 3. Scrape products on webshops
 4. Feed raw html (containing products) to LLM and classify enterprise

NACE Classification

- **Model:** `qwen3-6-35b-moe` via SSPCloud
- **Reference Data:** ÖNACE 2025 classification from Statistics Austria
 - Contains all ~37 five-digit G47 codes (G47.11–G47.83) with descriptions
 - Filtered to retail trade subset before classification
- **Extraction Process:**
 1. **Text Cleaning:** Remove HTML scripts, styles, and tags from scraped websites
 2. **Activity Description:** LLM extracts “primary business activity” in 1-3 sentences
 3. **Code Matching:** Compare activity description against all 37 G47 codes
 4. **Constrained Selection:** LLM receives full code list in prompt to ensure valid output
 5. **Output:** Predicted NACE code

NACE Classification - Results

- Started with 3,442 enterprises from SBR
- Identified websites for 1,217 enterprises

Website Detection

# Enterprise	# Websites	Coverage (%)
3442	1217	35.36

- 921 enterprises classified with LLM
- Top category: G47120 (17.4% = 160 enterprises)

Top 10 NACE Categories

NACE Code	Description	Enterprises	Share %
G47120	Sonstiger Einzelhandel mit Waren verschiedener Art	160	17.4
G47789	Einzelhandel mit sonstigen Neuwaren a.n.g.	84	9.1
G47520	Einzelhandel mit Metallwaren, Anstrichmitteln, Bau- und Heimwerkerbedarf	76	8.3
G47910	Vermittlungstätigkeiten für den Einzelhandel ohne ausgeprägten Schwerpunkt	72	7.8
G47110	Einzelhandel mit Waren verschiedener Art, Hauptrichtung Nahrungs- und Genussmittel	61	6.6
G47630	Einzelhandel mit Fahrrädern, Sport- und Campingartikeln	50	5.4

Thanks for listening!

Questions?